

เอกสารอ้างอิง

- [1] สมิต สัมภูกร. การประชุมที่เกิดประสิทธิผล : **Effective Meeting**. พิมพ์ครั้งที่ 1. กรุงเทพฯ : สายธาร. 2547.
- [2] นันทิยา หุตานุวัตร และณรงค์ หุตานุวัตร. **SWOT การวางแผนกลยุทธ์ธุรกิจชุมชน**. พิมพ์ครั้งที่ 5. กรุงเทพฯ : สถาบันชุมชนท้องถิ่นพัฒนา กรมวิทยาศาสตร์การแพทย์. 2545.
- [3] ปรีชา สิริโวหาร และสุทธธีรภย์ นาอะพันธุ์. 2547. “ระบบการจัดการการประชุม. ” ปรินญา นิพนธ์วิศวกรรมศาสตร์บัณฑิต ภาควิชาวิศวกรรมคอมพิวเตอร์, สถาบันเทคโนโลยีพระจอมเกล้าเจ้าคุณทหารลาดกระบัง.
- [4] การประชุม. 2549. **การประชุม**. [Online]. Available: th.wikipedia.org/wiki
- [5] การประชุมเชิงปฏิบัติการ. 2549. **การประชุมเชิงปฏิบัติการ**. [Online]. Available: <http://ilti.kku.ac.th/ams/57/963%20416/index-23.html>
- [6] International Organization for Standardization. 2549. **International Organization for Standardization**. [Online]. Available: <http://www.energythai.net/course/Tutorials/osi.html>
- [7] สถาปัตยกรรมรูปแบบ OSI. 2549. **สถาปัตยกรรมรูปแบบ OSI**. [Online]. Available: <http://irrigation.rid.go.th/rid15/ppn/Knowledge/Networks%20Technology/network6.htm>
- [8] ดร. วีระศักดิ์ ชิงฉาวร. 2546. **Java Programming Volume I**. พิมพ์ครั้งที่ 2. กรุงเทพฯ : สำนักพิมพ์ซีเอ็ดยูเคชั่น. 2546.
- [9] JDBC. 2549. **JDBC**. [Online]. Available: http://www.nextproject.net/article_detail.aspx?a_id=37

ภาคผนวก ก

JAVA API for Adobe Acrobat

เอกสารที่ใช้ในระบบสนับสนุนการประชุมเชิงปฏิบัติการจะเป็นไฟล์เอกสารชนิด Adobe Acrobat File (PDF) ดังนั้นระบบจะต้องสามารถเปิดไฟล์ PDF ขึ้นมาอ่านได้ ซึ่ง API ของจาวานั้นไม่มีคลาสที่รองรับการใช้งานเกี่ยวกับไฟล์ PDF จึงจำเป็นต้องหาไลบรารี (Library) จากที่อื่นมาช่วยจัดการเกี่ยวกับไฟล์ PDF ซึ่งไลบรารีที่นำมาใช้งานในระบบนี้มี 2 ตัว คือ

iText

เป็นไลบรารีที่ใช้ในการอ่านและเขียนไฟล์ PDF แต่ไลบรารี iText นี้ไม่สามารถแสดงข้อมูลที่อยู่ในไฟล์ PDF ในรูปแบบของเฟรม (Frame) หรือพาเนล (Panel) ได้ ถ้าต้องการแสดงผลในรูปแบบดังกล่าวจะต้องหาไลบรารีอื่นมาช่วย โดยไลบรารี iText นี้มีคลาสที่สำคัญ ดังต่อไปนี้

1. PdfReader ใช้สำหรับเปิดไฟล์ PDF เพื่อนำมาใช้งาน ซึ่งคลาสนี้เป็นการนำข้อมูลที่ได้ไปสร้างไฟล์อื่นในลักษณะการต่อสตริง (String)
2. Document ใช้สำหรับสร้างเอกสารการทำงานใหม่ ซึ่งสามารถกำหนดขนาดของเอกสารได้ด้วย
3. PdfWriter ใช้สำหรับสร้างอินสแตนซ์ (Instance) เพื่อเขียนเอกสารให้อยู่ในรูปแบบของเอาต์พุตสตรีม (Output Stream)
4. PdfImportedPage ใช้อ่านข้อมูลในไฟล์ PDF ที่ละหน้า เพื่อเขียนข้อมูลลงไปบนหน้าที่ต้องการ ซึ่งจะต้องทำงานร่วมกับ PdfWriter

สำหรับไลบรารี iText นี้สามารถค้นหาข้อมูลเพิ่มเติมและดาวน์โหลดได้ที่

<http://www.lowagie.com/iText/>

JPedal

เป็นไลบรารีที่ใช้ในการอ่านไฟล์ PDF และไลบรารีนี้ยังสามารถแสดงข้อมูลที่อยู่ในไฟล์ PDF ในรูปแบบของเฟรม (Frame) หรือพาเนล (Panel) ได้ จึงสะดวกต่อการใช้งาน แต่ไลบรารีนี้มีการใช้ในเชิงธุรกิจทำให้ไลบรารีที่ดาวน์โหลดฟรี (Freeware) ไม่สามารถใช้งานได้ครบทุกฟังก์ชัน (ในที่นี้จะมีเส้นสีแดง 2 เส้น คัดขวางในพาเนลที่ใช้แสดงผล) โดยไลบรารี JPedal นี้มีคลาสที่สำคัญ ดังต่อไปนี้

1. PdfDecoder ใช้ในการอ่านไฟล์ PDF เพื่อใช้ในการแสดงผล ซึ่งจะใช้งานคู่กับคลาส PdfStreamDecoder

2. PdfPanel เป็นคลาสที่สืบทอดมาจากคลาส Panel โดยคลาสนี้จะใช้แสดงผลไฟล์ PDF ที่อ่านจาก PdfDecoder
3. PdfStreamDecoder เป็นคลาสที่ใช้คู่กับ PdfDecoder โดยจะอ่านไฟล์ PDF ให้อยู่ในรูปของสตรีม (Stream)

PdfDecoder จะใช้เป็นคลาสหลักในการใช้งานไลบรารีนี้ ซึ่งภายในคลาส PdfDecoder มีเมธอดที่สำคัญในการใช้งาน ดังต่อไปนี้

ตารางที่ ก.1 เมธอดหลักที่ใช้งานในไลบรารี JPedal

เมธอด	รายละเอียด
setInset()	ใช้กำหนดจุดเริ่มต้นในการแสดงผลของข้อมูลที่อ่านเข้ามาว่าจะให้เริ่มต้นแสดงผลที่ตำแหน่ง x, y เท่าไรบนพาเนล
openPdfFile()	ใช้เปิดไฟล์ PDF ที่ต้องการ เพื่อนำมาแสดงผลในพาเนล โดยจะต้องใส่พารามิเตอร์ที่เป็นชื่อไฟล์ให้
setExtractionMode()	ใช้กำหนดโหมดการแสดงผล
decodePage()	ใช้เป็นตัวเลือกหน้าการแสดงผล
Repaint()	ใช้วาดภาพใหม่ลงพาเนล ซึ่งส่วนใหญ่จะเรียกใช้หลังจากที่เรียกเมธอด decodePage() เพื่อรีเฟรชหน้าการแสดงผลในพาเนล

สำหรับไลบรารี JPedal นี้สามารถค้นหาข้อมูลเพิ่มเติมและดาวน์โหลดได้ที่

<http://www.jpedal.org>

ภาคผนวก ข

Network Programming

เครือข่ายคอมพิวเตอร์ (Computer Network) คือ ระบบการสื่อสารระหว่างคอมพิวเตอร์จำนวนตั้งแต่สองเครื่องขึ้นไป การที่ระบบเครือข่ายมีบทบาทสำคัญมากขึ้นในปัจจุบันเป็นเพราะมีการใช้งานคอมพิวเตอร์อย่างแพร่หลาย จึงเกิดความต้องการที่จะเชื่อมต่อคอมพิวเตอร์เหล่านั้นถึงกัน เพื่อเพิ่มความสามารถของระบบให้สูงขึ้นและลดต้นทุนของระบบโดยรวมลง การโอนย้ายข้อมูลระหว่างกันเครือข่ายทำให้ระบบมีขีดความสามารถเพิ่มมากขึ้น การแบ่งการใช้ทรัพยากร เช่น หน่วยประมวลผล หน่วยความจำ หน่วยจัดเก็บข้อมูล โปรแกรมคอมพิวเตอร์และอุปกรณ์ที่มีราคาแพงซึ่งไม่สามารถจัดหามาให้ทุกคนได้ เช่น เครื่องพิมพ์ เครื่องกราดภาพ (scanner) ทำให้ลดต้นทุนของระบบลงได้

คอมพิวเตอร์ที่อยู่ในเครือข่ายเดียวกันสามารถติดต่อกันโดยใช้โปรโตคอล (Protocol) ที่เหมือนกัน ถึงแม้ว่าคอมพิวเตอร์นั้นจะมีโครงสร้างและระบบปฏิบัติการที่แตกต่างกันก็ตาม โปรโตคอลเป็นกฎเกณฑ์ที่กำหนดขั้นตอนในการติดต่อ การแลกเปลี่ยนข้อมูล รวมทั้งรูปแบบของข้อมูลที่ส่งไปมาระหว่างกัน โปรโตคอลถูกออกแบบและพัฒนาขึ้นมาเพื่อช่วยให้นักพัฒนาโปรแกรมสามารถพัฒนาโปรแกรมได้อย่างอิสระ ไม่ขึ้นกับภาษา ระบบปฏิบัติการ หรืออุปกรณ์สื่อสาร โดยโปรโตคอลที่เป็นที่รู้จักกันดี เช่น TCP, UDP, HTTP เป็นต้น

องค์การระหว่างประเทศเพื่อการทำให้เป็นมาตรฐาน (International Organization for Standardization - ISO) [6] ได้เล็งเห็นถึงปัญหาความเข้ากันไม่ได้ของระบบจากบริษัทผู้ผลิตต่างๆ ทั่วโลก ซึ่งเป็นความไม่สะดวกอย่างมากต่อผู้ใช้งาน จึงมีการพิจารณาร่างกรอบของมาตรฐานให้เป็นระบบเปิดเพื่อให้ระบบจากค่ายต่างๆ สามารถทำงานร่วมกันได้ เรียกว่า มาตรฐานการสื่อสารคอมพิวเตอร์ระบบเปิด (Open System Interconnection - OSI) ขึ้น ซึ่งใน OSI นี้จะยังไม่มีมาตรฐานใดๆ บรรจุอยู่ แต่เป็นเพียงกรอบของมาตรฐาน (Framework of Standards) ที่ระบุว่าควรแบ่งระบบเป็น 7 ชั้นการทำงาน และในแต่ละชั้นนั้นควรมีหน้าที่อะไรบ้าง เพราะฉะนั้นเมื่อระบบจากค่ายต่างๆ ถูกออกแบบขึ้นมาโดยยึดถือแนวทางของ OSI นี้ ระบบจากต่างบริษัท หรือต่างค่ายกันนั้นก็จะสามารถทำงานร่วมกันได้ ซึ่งทำให้กลุ่มผู้ใช้งานมีอิสระมากขึ้นในการพิจารณาเลือกใช้งานระบบต่างๆ เหล่านั้น

มาตรฐานการสื่อสารคอมพิวเตอร์ระบบเปิดแบ่งเป็น 7 ชั้นการทำงานนับจากชั้นบนสุดที่จะติดต่อโดยตรงกับผู้ใช้คือ

1. ชั้นแอปพลิเคชัน (Application Layer)
2. ชั้นพรีเซนเตชัน (Presentation Layer)

3. ชั้นเซสชัน (Session Layer)
4. ชั้นทรานสปอร์ต (Transport Layer)
5. ชั้นเน็ตเวิร์ก (Network Layer)
6. ชั้นดาต้าลิงก์ (Datalink Layer)
7. ชั้นฟิสิกัล (Physical Layer)

ซึ่งแต่ละชั้นการทำงานก็จะมีโปรโตคอล (protocol) ที่กำหนดขึ้นมาเพื่ออำนวยความสะดวกในการติดต่อสื่อสารกัน ขอบเขตของหน้าที่สำหรับแต่ละชั้นใน OSI นี้อาจแบ่งได้เป็น 2 กลุ่มใหญ่ๆ ดังนี้

- สำหรับ 3 ชั้นล่าง คือ ชั้นฟิสิกัล, ชั้นดาต้าลิงก์ และชั้นเน็ตเวิร์กนั้น จัดเป็นกลุ่มที่ขึ้นกับเทคโนโลยีก็จะเกี่ยวข้องกับการส่งผ่านข้อมูล การจัดข้อมูลเข้าสู่ชุดหรือหาเส้นทางสำหรับข่าวสารในเครือข่าย เป็นต้น โปรโตคอลในชั้นเหล่านี้จะเกี่ยวข้องกับเทคโนโลยีในการส่งผ่านข้อมูล เช่น สายนำสัญญาณต่างๆ, โทโปโลยี(topologies) คือรูปแบบการจัดวงจรข่ายคอมพิวเตอร์ หรือเทคนิคที่จะใช้ช่องข่าวสาร เป็นต้น กล่าวโดยสรุปคือในการทำงานที่ระดับ 3 ชั้นล่างนี้จะเกี่ยวข้องกับการปรากฏการณ์ที่เป็นกายภาพ เป็นส่วนของการสื่อสารข้อมูล (Data communication) ซึ่งก็คือ เกี่ยวข้องโดยตรงกับฝ่ายเทคนิคนั่นเอง
- สำหรับในชั้นที่ 4 คือ ชั้นทรานสปอร์ตนั้น เป็นตัวกลางแยกระหว่าง 3 ชั้นล่างและ 3 ชั้นบน อาจกล่าวได้ว่าสำหรับ 3 ชั้นล่างนั้นจะเป็นการสื่อสารข้อมูล ส่วน 3 ชั้นบนเป็นการประมวลผลข้อมูล โดยในชั้นทรานสปอร์ตนี้จะจัดการเกี่ยวกับปัญหาความผิดพลาด คือ ตรวจจับและแก้ไขความผิดพลาดที่อาจเกิดขึ้น จัดลำดับของข้อมูล ทำการระบุตำแหน่งสถานีปลายทาง และการมัลติเพล็กซ์ หน้าที่พื้นฐานที่สำคัญประการหนึ่งของชั้นทรานสปอร์ตนี้คือ แบ่งข่าวสารชุดยาวๆ จากชั้นบนคือชั้นเซสชันให้เป็นหน่วยที่เล็กลงเรียกว่า แพ็กเก็ต (Packet)
- ในชั้นที่ 5 คือ ชั้นเซสชันจะเกี่ยวกับหน้าที่ทางตรรกที่จำเป็นในการได้ตอบแลกเปลี่ยนข่าวสาร หน้าที่พื้นฐานสำหรับชั้นนี้คือ นำการบริการจากชั้นที่ 4 ซึ่งเป็นเพียงการย้ายข้อมูลดิบระหว่างเครื่องมาเพิ่มการบริการและทำให้แพ็กเก็ตของชั้นที่ 6 สามารถสร้าง ใช้ และยกเลิกการเชื่อมต่อระหว่างเครื่องได้ การเชื่อมต่อในระดับชั้นที่ 5 นี้ เรียกว่า เซสชัน กล่าวโดยสรุปคือ ในระดับเซสชันนี้ อาจมองได้ว่าเป็นจุดติดต่อระหว่างผู้ใช้ซึ่งในที่นี้หมายถึงผู้พัฒนาซอฟต์แวร์กับข่ายวงจรมันเอง
- ชั้นที่ 6 คือ ชั้นพรีเซนเตชัน ตัวอย่างของงานบริการในชั้นนี้ เช่น การเข้ารหัสลับ (encryption) การบีบอัดข้อมูล การส่งถ่ายแฟ้มข้อมูล เป็นต้น
- ชั้นที่ 7 คือ ชั้นแอปพลิเคชัน เป็นชั้นบนสุดของโมเดล ซึ่งเป็นการใช้งานที่ผู้ใช้จะคุ้นเคยอยู่แล้ว เช่น เวิลด์ ไวด์ เว็บ (World Wide Web) หรือการใช้อีเมล (e-mail) เป็นต้น

การทำงานของมาตรฐานการสื่อสารคอมพิวเตอร์ระบบเปิด

7 Application	โปรโตคอล Application	7 Application
6 Presentation	โปรโตคอล Presentation	6 Presentation
5 Session	โปรโตคอล Session	5 Session
4 Transport	โปรโตคอล Transport	4 Transport
3 Network	โปรโตคอล Network	3 Network
2 Data Link	โปรโตคอล Data Link	2 Data Link
1 Physical	สายสื่อสาร	1 Physical

รูปที่ ข.1 สถาปัตยกรรมรูปแบบ OSI

คอมพิวเตอร์ ก และคอมพิวเตอร์ ข มีโครงสร้างเป็น OSI ซึ่งมี 7 ชั้น เมื่อเครื่องคอมพิวเตอร์ ก พร้อมที่จะส่งสัญญาณข้อมูลไปยังเครื่องคอมพิวเตอร์ ข นั้น แต่ละชั้นการทำงานในเครื่องคอมพิวเตอร์ ก จะเสมือนกับว่ามีการสื่อสารกับชั้นการทำงานในระดับเดียวกันบนเครื่องคอมพิวเตอร์ ข ถึงแม้ว่าจะไม่มีการสื่อสารระหว่างชั้นการทำงานเหล่านี้เกิดขึ้นจริง แต่ชั้นการทำงานในระดับต่างๆ บนเครื่องคอมพิวเตอร์ทั้งคู่นั้นจะทำตามกฎเกณฑ์หรือโปรโตคอล (protocol) อย่างเดียวกัน เพื่อให้มั่นใจได้ว่าแต่ละชั้นการทำงานบนเครื่องคอมพิวเตอร์ฝ่ายผู้รับจะได้รับแพ็กเก็ตข้อมูลแบบเดียวกันกับแพ็กเก็ตข้อมูลที่รวบรวม แพ็กเก็ตข้อมูลจะเริ่มที่ระดับสูงสุดคือ ชั้นแอปพลิเคชันบนเครื่องคอมพิวเตอร์ ก และเคลื่อนลงมาที่ระดับชั้นจนมาถึงชั้นล่างสุดคือ ชั้นฟิสิกส์ การที่แพ็กเก็ตเคลื่อนผ่านจากระดับหนึ่งไปยังระดับถัดไปนั้น จะมีการกำหนดที่อยู่ การจัดรูปแบบของข้อมูล ซึ่งแต่ละชั้นการทำงานจะเป็นตัวจัดการและมีกระบวนการของตนเอง เมื่อแพ็กเก็ตเคลื่อนตัวลงมาถึงชั้นฟิสิกส์ก็จะถูกแปลงให้เป็นกระแสข้อมูลแบบอนุกรมและส่งผ่านสื่อกลางคือสายสัญญาณ ซึ่งเป็นชั้นการทำงานเดียวที่เครื่องคอมพิวเตอร์ ก สื่อสารกับเครื่องคอมพิวเตอร์ ข และเมื่อสัญญาณข้อมูลมาถึงเครื่องคอมพิวเตอร์ ข กระบวนการก็จะเริ่มทำในทางตรงข้าม คือจะทำการแยกแพ็กเก็ตออกจากผ่าน OSI ทั้ง 7 ชั้น แล้วส่งย้อนกลับขึ้นไปยังชั้นแอปพลิเคชันของเครื่องคอมพิวเตอร์ ข เมื่อแพ็กเก็ตเดินทางผ่านชั้นการทำงานระดับต่างๆ แต่ละชั้นการทำงานจะแยกข้อมูลข่าวสารตามที่อยู่ที่กำหนด และการจัดรูปแบบของแพ็กเก็ต จนเมื่อมาถึงชั้นการทำงานระดับสูงสุดคือ ชั้นแอปพลิเคชันก็จะเหลือเฉพาะข้อมูลที่เหมือนกับชั้นแอปพลิเคชันของเครื่องคอมพิวเตอร์ ก [7]

การใช้งานภาษาจาวากับการเขียนโปรแกรมด้านเครือข่ายนั้นจะใช้คลาส InetAddress ในการจัดการเกี่ยวกับเครือข่าย ซึ่งมีรายละเอียดดังต่อไปนี้

InetAddress

คลาส InetAddress จะอยู่ในแพ็คเกจ java.net ซึ่งเป็นคลาสที่ใช้เก็บและแสดง IP address บางกรณีอาจมีการเก็บชื่อโดเมน (Domain name) ด้วย คลาส InetAddress ใช้ได้กับโปรโตคอล TCP และ UDP โดย InetAddress นี้สามารถทำให้โปรแกรมใช้งานได้โดยไม่ขึ้นกับแพลตฟอร์ม (Platform) เนื่องจากแต่ละแพลตฟอร์มมีวิธีการเก็บ IP address แตกต่างกัน และถ้าในอนาคตมีการเปลี่ยนแปลงเวอร์ชันหรือรูปแบบของ IP address ใหม่ โปรแกรมที่ได้พัฒนาไว้แล้วก็ไม่ต้องแก้ไขใหม่ เพราะ InetAddress จะถูกปรับปรุงให้สนับสนุนเวอร์ชันของ IP address นั้นๆ

คลาส InetAddress ไม่มี public constructor แต่มี public static factory method สำหรับสร้างอินสแตนซ์ (instance) ของคลาส เช่น

`public static InetAddress getLocalHost() throws UnknownHostException;` เมธอดนี้จะส่งค่า InetAddress ของเครื่องที่ใช้งานกลับมาให้

`public String getHostName();` เมธอดนี้จะส่งค่าของชื่อโดเมนกลับมาในรูปแบบของสตริงชื่อค็อกเก็ต (String socket)

`public String getAddress();` เมธอดนี้จะส่งค่าของ IP address กลับมาในรูปแบบของสตริงชื่อค็อกเก็ต (String socket)

Socket

คลาส Socket จะอยู่ในแพ็คเกจ java.net ซึ่งเป็นคลาสที่ใช้สร้างชื่อค็อกเก็ต (Socket) ซึ่งใช้เป็นกลไกในการติดต่อไปยังเครื่องปลายทางที่พอร์ตใดพอร์ตหนึ่ง โดยเครื่องปลายทางอาจเป็นผู้รับหรือผู้ส่งก็ได้ รวมถึงใช้ในการสร้างสตรีม (Stream) สำหรับอ่านหรือเขียนข้อมูลไปยังเครื่องปลายทาง และใช้ปิดการติดต่อไปยังเครื่องปลายทาง

คอนสตรัคเตอร์ของคลาส Socket มีหลายตัว เช่น

`public Socket(String domain, int portNumber) throws UnknownHostException, IOException;`

`public Socket(InetAddress inet, int portNumber) throws IOException;`

`public Socket(String domainReceive, int portNumberReceive, String domainSender, int portNumberSender) throws IOException;`

`public Socket(InetAddress inetReceive, int portNumberReceive, InetAddress inetSender, int portNumberSender) throws IOException;`

คอนสตรัคเตอร์ 2 อันแรกจะต้องระบุชื่อของเครื่องปลายทางหรือ InetAddress และหมายเลขพอร์ต ส่วนคอนสตรัคเตอร์ 2 อันหลังนั้นมีการระบุข้อมูลเหมือน 2 ชนิดแรกแต่ต่างกันตรงที่ 2 อันหลังนี้จะต้องใส่ทั้งเครื่องปลายทางและต้นทาง การดักจับ IOException นั้นจะทำการกรณีที่พอร์ตของเครื่องปลายทางนั้นไม่ได้เปิดพอร์ตไว้ ส่วน UnknownHostException จะเกิดในกรณีที่ไม่มีชื่อโดเมนหรือ IP address นั้นอยู่จริง

เมื่อสร้างซ็อกเก็ตแล้วจะมีการติดต่อไปยังเครื่องปลายทางทันที โดยใช้โปรโตคอล TCP/IP หลังจากทั้งสองเครื่องติดต่อกันได้แล้ว เครื่องต้นทางสามารถขออินพุตสตรีม หรือเอาต์พุตสตรีมจากซ็อกเก็ต เพื่อใช้ในการอ่านและเขียนไปยังเครื่องที่อยู่ปลายทาง โดยจะต้องมีโปรโตคอลที่ใช้ในการตกลงกันระหว่างเครื่องทั้งสองว่าจะรับส่งข้อมูลอย่างไร ข้อมูลมีรูปแบบใด เป็นต้น ในแต่ละพอร์ตที่มีการใช้งานจะมีการกำหนดโปรโตคอลไว้ตายตัว เพื่อให้สามารถติดต่อกับพอร์ตนั้นๆ ได้ง่าย หมายเลขพอร์ตต้องเป็นจำนวนเต็มตั้งแต่ 1 – 1024 จะถูกกำหนดให้เป็นพอร์ตที่มีโปรโตคอลมาตรฐานไว้แล้ว เช่น พอร์ตหมายเลข 21 ใช้โปรโตคอล FTP พอร์ตหมายเลข 80 ใช้โปรโตคอล HTTP พอร์ตหมายเลข 25 ใช้โปรโตคอล SMTP เป็นต้น ดังนั้นหากมีการเปิดพอร์ตใช้งานเอง ควรเลือกหมายเลขที่สูงกว่า 1024 ขึ้นไป และควรตรวจสอบก่อนการใช้งานทุกครั้งด้วยว่าพอร์ตหมายเลขนั้นมีการใช้งานอยู่ก่อนแล้วหรือไม่

ServerSocket

ใช้ในการส่งข้อมูลระหว่างเครื่องสองเครื่องผ่านทางซ็อกเก็ต โดยจะต้องมีเครื่องใดเครื่องหนึ่งเป็นผู้รับการติดต่อ (Server) และอีกเครื่องหนึ่งเป็นผู้ติดต่อ (Client) ในภาษาจาวามีคลาส ServerSocket ซึ่งมีกลไกสำหรับการรอรับการเชื่อมต่อจากเครื่องไคลเอนท์ที่พอร์ตใดพอร์ตหนึ่ง เมื่อมีการติดต่อมาจากไคลเอนท์ คลาสนี้จะทำการเชื่อมการติดต่อและสร้างซ็อกเก็ตให้เครื่องที่เป็นเซิร์ฟเวอร์ใช้ในการรับส่งข้อมูลกับเครื่องที่เป็นไคลเอนท์ ในคลาสนี้สามารถสร้างซ็อกเก็ตได้เพียงซ็อกเก็ตเดียวต่อไคลเอนท์หนึ่งเครื่อง ดังนั้นถ้าต้องการส่งข้อมูลแบบกระจาย (Broadcast) ซึ่งเป็นการส่งข้อมูลชุดหนึ่งไปให้หลายๆ ไคลเอนท์โดยส่งเพียงครั้งเดียว กรณีนี้จะต้องใช้คลาส MulticastSocket

คอนสตรัคเตอร์ของคลาส ServerSocket มีหลายตัว เช่น

public ServerSocket() throws IOException; ใช้สร้างซ็อกเก็ตโดยที่ยังไม่กำหนดหมายเลขพอร์ต
public ServerSocket(int) throws IOException; ใช้สร้างซ็อกเก็ตโดยมีการกำหนดหมายเลขพอร์ต
public ServerSocket(int, int) throws IOException; ใช้สร้างซ็อกเก็ตโดยมีการกำหนดหมายเลขพอร์ต และความยาวของแถวคอย (Queue) ในการเชื่อมต่อ

`public ServerSocket(int, int, InetAddress) throws IOException;` ใช้สร้างซ็อกเก็ตโดยมีการกำหนดหมายเลขพอร์ต ความยาวของคิวในการเชื่อมต่อ และ `InetAddress` ของเครื่องที่รอรับการติดต่อ (คอนสตรัคเตอร์ตัวที่ 2 และ 3 จะใช้ `localhost` แทน)

ระบบปฏิบัติการจะจัดเก็บแถวคอย (Queue) ในการเชื่อมต่อแบบเข้าก่อนออกก่อน (First In – First Out) โดยทั่วไปแถวคอยในการเชื่อมต่อเหล่านี้มีความยาวที่เป็นค่าปริยาย (Default) และค่าสูงสุดแตกต่างกันไปขึ้นอยู่กับระบบปฏิบัติการ

เมื่อมีการเชื่อมต่อเข้ามาที่พอร์ตใดพอร์ตหนึ่ง ระบบปฏิบัติการจะจัดเก็บการเชื่อมต่อที่ลงในแถวคอยของพอร์ตหมายเลขนั้นๆ ถ้ามีการเชื่อมต่อเข้ามาจำนวนมากๆ อย่างรวดเร็ว จะทำให้เซิร์ฟเวอร์ทำงานไม่ทัน ส่งผลในแถวคอยเต็ม การเชื่อมต่อที่เข้ามาระหว่างนั้นจะสูญหายไปและไม่ได้รับบริการ นักพัฒนาโปรแกรมไม่สามารถแก้ปัญหาที่เกิดขึ้นนี้ได้ เนื่องจากระบบปฏิบัติการเป็นตัวจัดการงานในส่วนนี้ จึงทำได้เพียงกำหนดความยาวของแถวคอยให้ยาวมากขึ้นเท่านั้น

เมธอดที่จำเป็นในการทำงานที่สำคัญอีกเมธอดหนึ่ง คือ

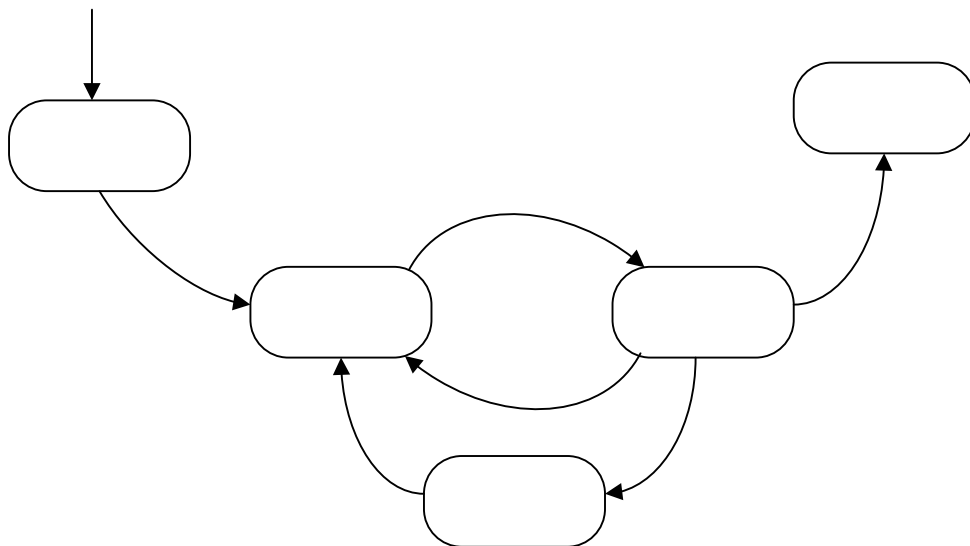
`public Socket accept() throws IOException;` ใช้ในการรอรับการเชื่อมต่อจากไคลเอนท์

ภาคผนวก ค

Thread

ระบบปฏิบัติการที่มีความสามารถในการทำงานแบบมัลติทาสก์กิ้ง (Multitasking) นั้นจะทำงานโดยแบ่งโปรเซส (Process) ออกเป็นหน่วยย่อยๆ เรียกว่า เธรด (Thread) ดังนั้นใน 1 โปรเซส จะมีได้หลายเธรด ซึ่งทำให้โปรเซสนั้นสามารถทำงานได้หลายอย่างพร้อมๆ กัน ส่งผลให้โปรแกรมทำงานได้เร็วขึ้น และเป็นการใช้งานหน่วยประมวลผลได้อย่างเต็มประสิทธิภาพมากขึ้นด้วย

หน้าที่ในการจัดการการทำงานของเธรดนั้นเป็นหน้าที่ของระบบปฏิบัติการ ดังนั้นนักพัฒนาโปรแกรมจึงทำได้เพียงแค่การสร้างเธรดให้ทำตามหน้าที่ที่ต้องการ และควบคุมให้เธรดเหล่านั้นเริ่มทำงาน หรือหยุดทำงาน ในระหว่างที่เธรดทำงานก็สามารถออกคำสั่งให้เธรดหยุดรอ และเริ่มการทำงานอีกครั้งเมื่อถึงเวลาที่กำหนดได้วัฏจักร (Life cycle) [8] ของเธรดแสดงดังรูปที่ ค.1



รูปที่ ค.1 วัฏจักรของเธรด

วัฏจักรของเธรด

เมื่อมีการสร้างเธรดขึ้นมาโดยใช้คีย์เวิร์ด `new()` แล้วระบบปฏิบัติการจะกำหนดค่าเริ่มต้นให้กับตัวแปรบางตัว แต่ในขั้นตอนนี้เธรดจะยังไม่เริ่มการทำงานจนกว่าจะได้รับคำสั่ง `start()` เธรดนั้นจึงจะเปลี่ยนไปเป็นสถานะ `ready` คือ รออยู่ในแถวคอยของเธรดจนกว่าจะถูกเลือกออกไปทำงานในหน่วยประมวลผล โดยปกติแล้วเธรดจะเปลี่ยนสถานะจาก `ready` ไปเป็น `running` เร็วมาก หากไม่มีเธรดในสถานะ `running` ระบบปฏิบัติการจะเลือกเธรดขึ้นมาจากแถวคอยของ เธรดที่อยู่

ในสถานะ ready ไปทำงาน กฎเกณฑ์ที่ใช้ในการเลือกนี้ขึ้นกับระบบปฏิบัติการ ไม่ได้มีการกำหนดในภาษาจาวา

เมื่อเธรดเข้ามาอยู่ในสถานะ running แล้วจะเปลี่ยนสถานะต่อไปได้อีก 3 สถานะ คือ

- สถานะ die เมื่อเธรดนั้นได้รับคำสั่ง stop() ก็จะคืนพื้นที่ในหน่วยความจำให้กับระบบ ไม่สามารถกลับเข้าสู่สถานะ ready ได้อีก
- สถานะ ready เมื่อเธรดนั้นได้รับคำสั่ง yield() หรือถูกขัดจังหวะการทำงาน ส่งผลให้เธรดนั้นหยุดการทำงานชั่วคราว และเธรดอื่นที่คอยอยู่ในสถานะ ready ได้เข้ามาทำงานแทน ซึ่งเธรดนั้นจะกลับเข้ามาทำงานอีกครั้งเมื่อถูกเลือก
- สถานะ block เมื่อเธรดนั้นได้รับคำสั่ง wait(), suspend() หรือ sleep() จากโปรแกรม หรือทำการติดต่อกับ i/o หรือรอใช้ทรัพยากรของเครื่องที่มีเธรดอื่นใช้งานอยู่ เธรดที่อยู่ในสถานะนี้จะกลับไปอยู่ในสถานะ ready อีกครั้งก็ต่อเมื่อได้รับคำสั่ง notify(), resume() หรือเมื่อหมดเวลาอันเนื่องจากการใช้คำสั่ง sleep() หรือเมื่อเสร็จสิ้นการติดต่อกับ I/O หรือได้รับทรัพยากรของระบบที่รอใช้งานอยู่

ภาคผนวก ง

Vector

อาร์เรย์ (Array) ในภาษาจาวจะมีขนาดคงที่ กล่าวคือ เมื่อสร้างอาร์เรย์ขึ้นมาแล้วจะไม่สามารถเปลี่ยนแปลงขนาดพื้นที่ของอาร์เรย์ให้เพิ่มขึ้นหรือลดลงอีกได้ ในการใช้งานจริงจึงต้องกำหนดขนาดของอาร์เรย์ให้มีขนาดใหญ่กว่าขนาดที่ต้องการใช้งานจริง ซึ่งการกระทำลักษณะนี้เป็นการสิ้นเปลืองหน่วยความจำ และอาจเกิดความผิดพลาดได้ในกรณีที่ปริมาณข้อมูลมากกว่าขนาดของอาร์เรย์ที่เตรียมไว้

ปัญหาที่เกิดขึ้นสามารถแก้ไขได้โดยการใช้เวกเตอร์ (Vector) [8] ช่วย โดยเวกเตอร์นี้จะใช้สำหรับเก็บกลุ่มของข้อมูลที่ไม่ทราบขนาดที่แท้จริง หรือกล่าวอีกนัยหนึ่งได้ว่าเป็นอาร์เรย์ที่สามารถเปลี่ยนแปลงขนาดพื้นที่ให้เพิ่มขึ้นหรือลดลงได้อัตโนมัติขณะที่โปรแกรมทำงาน

คลาสเวกเตอร์มีคอนสตรัคเตอร์ (Constructor) 3 ตัว คือ

1. `public Vector();` ใช้สร้างเวกเตอร์ที่มีขนาดเริ่มต้นเป็นค่าปริยาย(default) คือ 10
2. `public Vector(int initialCapacity);` ใช้สร้างเวกเตอร์ที่มีขนาดเริ่มต้นเท่ากับค่าของตัวแปร `initialCapacity`
3. `public Vector(int initialCapacity,int capacityIncrement);` ใช้สร้างเวกเตอร์ที่มีขนาดเริ่มต้นเท่ากับค่าของตัวแปร `initialCapacity` และจะเปลี่ยนขนาดครั้งละ `capacityIncrement`

ตารางที่ ง.1 เมธอดที่สำคัญของคลาส Vector

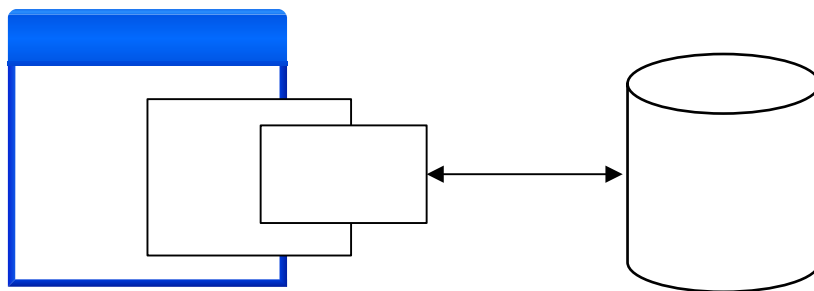
เมธอด	รายละเอียด
<code>add(int index, E element)</code>	แทรกค่าของตัวแปร <code>element</code> ลงในเวกเตอร์ ณ ตำแหน่งที่กำหนดไว้ที่ตัวแปร <code>index</code>
<code>addElement(E obj)</code>	เพิ่มอิลลิเมนต์ (<code>element</code>) ต่อท้ายเวกเตอร์ และเพิ่มขนาดอีก 1
<code>clear()</code>	ลบอิลลิเมนต์ทั้งหมดออกจากเวกเตอร์
<code>elementAt(int index)</code>	คืนค่าอิลลิเมนต์ในตำแหน่งที่ต้องการ
<code>firstElement()</code>	คืนค่าอิลลิเมนต์ในตำแหน่งแรกสุด
<code>insertElementAt(E obj, int index)</code>	เพิ่มอิลลิเมนต์ในเวกเตอร์ ณ ตำแหน่งที่ต้องการ
<code>isEmpty()</code>	ตรวจสอบว่าเวกเตอร์นั้นมีอิลลิเมนต์หรือไม่
<code>lastElement()</code>	คืนค่าของอิลลิเมนต์ตัวสุดท้ายในเวกเตอร์
<code>remove(Object o)</code>	ลบอิลลิเมนต์ออกจากเวกเตอร์ในตำแหน่งที่ต้องการ
<code>size()</code>	คืนค่าของจำนวนอิลลิเมนต์ที่อยู่ในเวกเตอร์

ภาคผนวก จ

JAVA Database Connection (JDBC)

JDBC เป็นจาวาเอพีไอ (JAVA API) สำหรับการประมวลผลคำสั่ง SQL ซึ่งเป็นคลาสและอินเทอร์เฟซที่เป็นภาษาจาวา JDBC [9] ได้จัดเตรียมเอพีไอมาตรฐานสำหรับเป็นเครื่องมือในการพัฒนา แอปพลิเคชันด้านฐานข้อมูล (Database Application) ในการเขียนโค้ดโปรแกรมภาษาจาวาโดยใช้ JDBC นั้นสามารถติดต่อกับฐานข้อมูลได้เกือบทุกประเภท เช่น Sysbase, Oracle, MS SQL, Informix, Access, MySQL

การพัฒนาแอปพลิเคชันด้านฐานข้อมูลโดยใช้จาวา สามารถนำไปดำเนินงาน (Run) ได้หลายแพลตฟอร์ม (Platform) ทำให้ทำการพัฒนาแอปพลิเคชันสามารถใช้คุณสมบัติของภาษาจาวาได้อย่างเต็มที่ JDBC ถูกประยุกต์ใช้งานในหลายด้าน เช่น ในเว็บเพจที่มีแอปเพล็ต (Applet) ในการดำเนินงานแบบรีโมทดาต้าเบส (Remote Database) หรือมีการใช้ JDBC ในการติดต่อกับฐานข้อมูล



รูปที่ จ.1 แสดงการติดต่อระหว่างโปรแกรมและระบบฐานข้อมูล

ข้อดีของ JDBC

1. ใช้งานง่าย การตั้งค่าไม่ยุ่งยาก
2. สามารถทำงานร่วมกับภาษาจาวาได้เป็นอย่างดี เพราะไครเวอร์ที่เขียนขึ้นนั้นส่วนมากจะพัฒนาจากภาษาจาวาและมีการคอมไพล์ไฟล์เป็นคลาส ทำให้ใช้งานง่าย
3. สามารถติดต่อกับฐานข้อมูลได้เกือบทุกประเภทซึ่งส่วนใหญ่จะเป็นฐานข้อมูลแบบเชิงสัมพันธ์
4. ไฟล์ไครเวอร์มีขนาดเล็ก
5. สามารถใช้งานได้ทั้ง สถาปัตยกรรมแบบทิวเทียร์ (Two-Tier) และทรีเทียร์ (Three-Tier)
6. สามารถใช้งานได้หลายระบบปฏิบัติการ
7. การใช้งานจะเป็นแบบเชิงวัตถุ (Object-Oriented)
8. ไครเวอร์แต่ละตัวมีมาตรฐานใกล้เคียงกัน

การทำงานที่สำคัญของ JDBC

1. ทำการติดต่อกับ Data Base
2. ตั้งคำสั่ง SQL ไปยังฐานข้อมูล
3. รับผลลัพธ์จากการใช้คำสั่ง SQL แล้วทำการประมวลผล

ลักษณะที่สำคัญของ JDBC

JDBC เป็นอินเทอร์เฟซระดับล่าง (low-level Interface) จึงเรียกใช้คำสั่ง SQL ได้โดยตรง สามารถใช้งานได้มีประสิทธิภาพ มีขนาดเล็กและง่ายต่อการใช้งาน แต่ก็ได้มีการออกแบบสำหรับการใช้งานแบบอินเทอร์เฟซระดับบน (High-level Interface) ด้วย โดยมีความสามารถเพิ่มเติมดังนี้

1. JDBC ต้องการคำสั่ง SQL ที่จะทำการส่งเป็นสตริง (String) ไปยังเมธอดของภาษาจาวา ทำให้นักพัฒนาโปรแกรมสามารถใช้ตัวแปรในการรับส่งคำสั่ง SQL ได้
2. มีการแปลงตารางของฐานข้อมูลแบบ relation ไปเป็นคลาสของจาวา เรียกว่า Object/Relational ซึ่งแต่ละแถวของตารางจะถูกแปลงไปเป็นอินสแตนซ์ของคลาส และแต่ละคอลัมน์จะเป็นเหมือนแอททริบิวต์ (attribute) ของคลาส

การเปรียบเทียบระหว่าง JDBC และ ODBC

Microsoft ODBC API สามารถใช้แอปพลิเคชันในการติดต่อกับฐานข้อมูลได้เกือบทุกประเภทเช่นกัน แต่ในภาษาจาวาจะไม่ใช้ ODBC ในการติดต่อฐานข้อมูล เนื่องจาก

1. ODBC ไม่มีความสะดวกในการใช้งานได้โดยตรงจากจาวา เพราะมีการใช้อินเทอร์เฟซที่เป็นภาษา C ในการเรียกคำสั่งจากจาวา แล้วแปลงเป็นโค้ดภาษา C จะทำให้เกิดผลกระทบหลายเรื่อง เช่น ด้านความปลอดภัย การใช้งาน ความเสถียรของแอปพลิเคชัน
2. ตัวแปรที่มีการแปลงจาก ODBC API ไปเป็นจาวามืออาชีพ จะทำให้เกิดปัญหา เช่น จาวาไม่มีพอยท์เตอร์ ในขณะที่ภาษาซีมีการใช้งานพอยท์เตอร์
3. ODBC ยากต่อการเรียนรู้ เพราะมีการผสมผสานคุณสมบัติทั้งง่ายและยากเข้าไว้ด้วยกัน มีการนำเอาฟังก์ชันต่างๆ มารวมกันไว้ทำให้ต้องศึกษาทั้งหมด แต่ JDBC มีการออกแบบมาเพื่อให้ใช้งานได้ง่ายขึ้น โดยแยกฟังก์ชันต่างๆ ออกจากกัน
4. จาวาและ JDBC จะมีการใช้งานร่วมกันได้เพราะต่างก็เขียนขึ้นมาด้วยภาษาจาวา และจะถูกบรรจุอยู่ในแอปพลิเคชันอัตโนมัติ ซึ่งจะมีความปลอดภัยกับข้อมูลที่ใช้งานในระบบเครือข่าย ซึ่งต่างจาก ODBC ซึ่งต้องมีการติดตั้งไดรเวอร์ลงในทุกๆ เครื่องที่มีการใช้งาน

ประเภทของ JDBC Driver

1. JDBC - ODBC Bridge เป็นไดรเวอร์ของ JDBC ที่ใช้ในการติดต่อการทำงานผ่าน ODBC ซึ่งจะมีการใช้ ODBC ในการติดต่อกับฐานข้อมูลก่อนไม่ว่าจะอยู่ในเครื่องเดียวกันหรือติดต่อผ่านเซิร์ฟเวอร์ แล้วจึงใช้ JDBC ไปติดต่อกับ ODBC อีกครั้งหนึ่ง
2. Native API partly-Java Driver ไดรเวอร์ประเภทนี้จะเปลี่ยนการเรียก JDBC ไปเรียกเอพีไอที่เป็นไคลเอนท์ของ Oracle, Sysbase หรือ DBMS ตัวอื่นๆ ซึ่งจะคล้ายกับสะพานเชื่อมต่อไปยังฐานข้อมูล
3. JDBC-Net Java Driver ไดรเวอร์ประเภทนี้จะเปลี่ยนการเรียก JDBC ไปยัง DBMS - independent net protocol ซึ่งหลังจากการเปลี่ยนไปเป็น DBMS Protocol โดยเซิร์ฟเวอร์ ซึ่งเซิร์ฟเวอร์นี้สามารถติดต่อเข้ากับจาวาไปยังฐานข้อมูลหลายประเภทได้ซึ่งจะสามารถใช้งานผ่านทางอินเทอร์เน็ตและอินทราเน็ต
4. Native-protocol pure Java Driver ไดรเวอร์ประเภทนี้จะเปลี่ยนการเรียก JDBC ไปเป็นโปรโตคอลของระบบเครือข่าย ซึ่งจะถูกใช้โดย DBMS โดยมีการเรียกใช้งานโดยตรงจากไคลเอนท์ไปยังเซิร์ฟเวอร์ของ DBMS

บริษัท JavaSoft ได้กำหนดคุณลักษณะของผลิตภัณฑ์ของ JDBC component ไว้ดังนี้

1. การจัดการไดรเวอร์ JDBC เป็นหัวใจหลักของสถาปัตยกรรมของ JDBC ซึ่งต้องมีขนาดเล็กและง่าย ซึ่งต้องมีฟังก์ชันพื้นฐานในการติดต่อไปยังแอปพลิเคชันของจาวา
2. การทดสอบไดรเวอร์ JDBC จะเป็นตัวตัดสินใจว่า JDBC นี้สามารถใช้งานร่วมกับแอปพลิเคชัน และมีการทำงานได้หรือไม่
3. JDBC-ODBC Bridge สามารถใช้ไดรเวอร์ ODBC คล้ายกับไดรเวอร์ JDBC

การใช้งาน

จะต้อง import java.sql.* ก่อน ซึ่งแพ็คเกจนี้จะบรรจุคลาสต่างๆ ที่สนับสนุนการทำงานกับฐานข้อมูล โดยคลาสหลักๆ ที่เกี่ยวข้องกับการทำงาน มีดังนี้

- connection ใช้ในการสร้างการเชื่อมต่อระหว่างโปรแกรมกับตัว DBMS โดยจะต้องเลือกไดรเวอร์ให้เหมาะสมกับฐานข้อมูลที่ต้องการใช้
- statement ใช้ในการกำหนดคำสั่ง sql ต่างๆ ลงไปเพื่อรอการ execute query
- resultSet ใช้เป็นผลลัพธ์ของการ query statement ของคลาส Statement